Legacy Python Application

Migration Knowledge & System Specification Document

1. Document Purpose

This document describes a legacy Python application that manages users, orders, customer information, configuration, and report generation. It is intentionally written as an older technical specification so that an automated code migration agent can analyze the document and identify modernization requirements.

The migration system should retrieve relevant sections when migrating individual modules. Business rules and observable behavior must be preserved even when the implementation is modernized.

2. Application Overview

The application is a small command-line business management system developed using Python. It contains a user management module, a local JSON-based persistence layer, an order processing module, and a configuration module.

The original application was developed using older Python practices. Several parts of the system use explicit file open and close operations, broad exception handling, manual loops, mutable data structures, and procedural helper functions.

3. User Management Module

The User component represents a customer of the application. Every user contains a unique identifier, a name, an email address, and a creation timestamp.

User attributes:

• user_id — unique customer identifier
• name — customer's display name
• email — customer's email address
• created_at — timestamp indicating when the object was created

The legacy implementation exposes individual getter methods such as get_id(), get_name(), and get_email(). The print_user() operation writes a formatted user record to the console.

4. User Repository

The UserRepository component provides persistent storage for users. Instead of a relational database, the legacy application stores user records in a JSON file named users.json.

When a new user is saved, the repository checks whether the JSON file exists. If it exists, the existing list of users is loaded. The new user's information is appended to the list and the complete list is written back to the file.

The legacy implementation manually opens files and closes them in finally blocks. A migration should consider replacing this pattern with Python context managers.

The find_user operation searches for a user by identifier. If the JSON file does not exist, the operation returns no result. If a matching identifier is found, a User object is reconstructed and returned.

5. User Service

UserService contains application-level operations involving users. It receives a repository object and uses that repository to retrieve users.

The get_user operation requests a user from the repository. If no user is found, the service prints 'User not found' and returns no result. If a user exists, the service prints the user information and returns the User object.

The find_all_users operation receives a collection of user identifiers. It checks each identifier and adds successfully retrieved users to a result collection. Unknown identifiers are skipped rather than causing the entire operation to fail.

6. Order Management Module

The Order component represents a customer's purchase. An order contains an order identifier, a customer identifier, a list of purchased items, and a numeric total.

Order behavior:

• A newly created order has an empty item list.
• The initial total is zero.
• Adding an item appends the item description to the item list.
• The item's price is added to the current total.
• The order can print its identifier, items, and total.

The migration must preserve the total calculation exactly. For example, an order containing items priced at 2500, 1000, and 500 must have an original total of 4000.

7. Discount Rules

The legacy application calculates discounts using a separate procedural function called calculate_discount.

Premium customers receive a 20 percent discount. Regular customers receive a 10 percent discount. Any other customer type receives no discount.

The discount is calculated from the original order total. The final total is calculated by subtracting the discount from the original total.

8. Order Summary

The generate_order_summary operation creates a dictionary containing the order identifier, customer identifier, items, original total, discount amount, and final total.

The summary is later serialized into a JSON file. The migration may improve the representation of this data, but the same information must remain available.

9. Configuration Module

The configuration component stores application-wide constants. The legacy system defines the application name as LegacyMigrationApp, the environment as production, and the maximum retry count as 3.

A print_config operation displays these values. Configuration values should not be accidentally changed during migration.

10. Main Application Workflow

The main workflow creates a repository connected to users.json and then creates a UserService using that repository.

Two users are created and stored: user 101 with the name Alice and user 102 with the name Bob. The application then searches for users 101, 102, and 999. The unknown user 999 must be ignored.

An order with identifier 5001 is created for customer 101. Three products are added: Keyboard priced at 2500, Mouse priced at 1000, and USB Cable priced at 500. The original order total is therefore 4000.

The customer is treated as a premium customer, so a 20 percent discount is applied. The discount is 800 and the final total is 3200.

The resulting order summary is written to order_summary.json and the application configuration is printed.

11. Legacy Implementation Characteristics

The following characteristics are intentionally present in the legacy design:

1. Explicit file opening and closing.
2. Broad Exception handling in service and persistence code.
3. Manual list construction and indexed loops.
4. Getter methods for simple object attributes.
5. Mutable dictionaries used as data-transfer objects.
6. String concatenation for formatted output.
7. Procedural functions mixed with object-oriented components.
8. Local JSON files used as a persistence mechanism.
9. Minimal type information.
10. Configuration values stored as class constants.

12. Migration Requirements

The migration agent should modernize the implementation while preserving behavior. Potential improvements include Python type hints, dataclasses for simple data models, pathlib for filesystem operations, context managers for file handling, f-strings for formatting, clearer exception handling, and more readable collection operations.

The agent should not change business rules merely because the implementation is being modernized. In particular, the premium discount must remain 20 percent, the regular discount must remain 10 percent, and unknown customer types must receive no discount.

13. Retrieval Test Scenarios

This document can be used to test semantic retrieval in the migration system. Useful queries include the following:

Query	Expected relevant information
How are users stored?	User Repository / JSON persistence
How is a user retrieved?	find_user behavior
What happens when a user does not exist?	UserService behavior
What are the order calculation rules?	Order Management
How is the premium discount calculated?	Discount Rules
What is the expected final total for order 5001?	Main workflow / discount rules
How are files handled in the legacy system?	Legacy implementation characteristics
What should be modernized?	Migration requirements
Which business rules must be preserved?	Migration requirements / Discount Rules

14. Expected Migration Outcome

A successful migration should produce a cleaner, maintainable Python implementation while retaining the original architecture's core responsibilities and all important business behavior. The migration agent should be able to retrieve the relevant documentation sections rather than relying only on adjacent text.

This document is intentionally suitable for testing chunking, embedding generation, vector search, semantic retrieval, and context-aware migration.